

## 5., A szoftver fejlesztés biztonsági kérdései – a biztonságos kód

148

- ▶ „Stand alone” rendszerek → a szoftverhiba, biztonsági rés helyi hatású.
- ▶ „Hálózatosítás” → a hiba kiterjedhet az intézmény hálózatában lévő eszközökre.
- ▶ „Internetesítés” a hiba bárhonnán, bármikor kihasználható.
- ▶ A KGB visszahozta a írógépeket ...

148

### A Stuxnet „story”

149

- ▶ Speciális, igen komplex vírus, amely 3 rétegen keresztül működik Windows oprendszer, a Siemens Simatic folyamatfelügyeleti rendszer, PLC<sup>3</sup> programok.
- ▶ Kifejezetten az iráni urándúsító centrifugáinak inverteres motormeghajtóira szállt rá a féreg.
- ▶ 2010. november 16-án Irán leállította az urándúsítóit, miután a centrifugák több, mint 20%-a megsemmisült a Stuxnet tevékenysége nyomán, azaz a kártevő elérte a célját.
- ▶ Azóta az első cyber-fegyverként tartják számon
- ▶ Nagy team, szoftver tervezés, sok pénz kellett hozzá, ma sem tudni pontosan, hogy kik írták ...

149

## Biztonsági szempontból nézve a támadás tárgya lehet:

150

- ▶ Szoftverhiba, ténylegesen hibásan működő funkcionalitás (a komplexitás miatt előfordul,)
- ▶ Hibás inputszűrés (az első védelmi rendszer hibája)
- ▶ Feledékenységből „nyitva hagyott ajtó” (több feltétel együttállását nem vizsgáljuk, nem szűrjük ki).
- ▶ Szándékosan „nyitva hagyott hátsóajtó” (későbbi illegális belépésre, vagy magunknak vészmegoldásként felhasználva).

150

## Kódolási hibák száma átlagosan

151

- ▶ Pontos adat nincsen, (a cégek „szégyenlősek”) az alábbi közelítő adatok érhetők el:
- ▶ Ipari átlag: 15-50 hiba / 1000 sor, befolyásoló tényezők: a fejlesztési módszertan, a tesztelés minősége, a fejlesztő képességei, stb.
- ▶ Microsoft termékek: 10-20 hiba / 1000 sor a belső tesztelés előtt, 0.5 hiba / 1000 sor a kiadás után
- ▶ Legjobb módszerek: pl.: Harlan Mills: „cleanroom development”: 3 hiba / 1000 sor tesztelés előtt, 0.1 hiba / 1000 sor kiadás után

151

## 5.1. A szoftverhibák osztályozása

152

- ▶ A legjobb módszertannal és teszteléssel is maradnak hibák a szoftverekben, amelyek utólag derülnek ki.
- ▶ A hibák biztonsági szempontból különböző hatásúak lehetnek. A hibákat hatásuk alapján a következő csoportokba sorolhatjuk:
  1. **Biztonsági hatásuk nincs**, csupán a felhasználót idegesíti a hiba, inkább funkcionális hiba, mint biztonsági

152

2. **Alacsony hatású biztonsági hiba**: nem igényel azonnali beavatkozást pl. információszivárgás (beszédes hibaüzenet: osztálynév vagy adatbázis táblanév kiszivárog), biztonsági szempontból önállóan és közvetlenül nem jelent kockázatot
3. **Közepes hatású biztonsági hiba**: közvetlen biztonsági kockázatot jelent és sürgős beavatkozást igényel pl. Cross-site scripting sérülékenység (megtévesztéssel együtt alkalmazva kliensoldali adatok szerezhetők meg webbrower-en keresztül) (Alkotmánybíróság weboldalán így írták át az alaptörvényt)

153

153

4. **Magas hatású biztonsági hiba:** azonnali beavatkozást igényel, pl. SQL injection (a támadó a felhasználók bizalmas adataihoz férhet hozzá a szerveren a szerver oldali kód hibája miatt)

154

5. **Kritikus biztonsági hiba:** pl. Buffer overflow az Operációs rendszer valamelyik api-jában ( a hibán keresztül távolról kód futtatható az operációs rendszeren, a sérülékeny szoftvert emberek milliói használják naponta)

154

## Szoftverhibák helye

155

- Egyedi szoftverekben vagy weboldalakon: A hibás szoftverkód egy egyedi fejlesztés eredménye, a hibás kód máshol nem fordul elő. (pl. hibás php script egy egyedi weboldalon)
- Tömegtermékekben: A hibás szoftverkód egy hivatalosan kiadott szoftvertermékben van jelen pl. Weboldalak CMS szoftverében, Webszerverek kódjában, Asztali alkalmazásokban, Mobile app-okban, stb.



**Sérülékenységi adatbázisok**

155

## Sérülékenységi adatbázisok

156

- ▶ A tömegesen használt és hivatalosan kiadott termékekben lévő hibákat nyilvántartják és folyamatosan regisztrálják:
- ▶ Computer Emergency Response Team (CERT), 1988-ban alapította a DARPA (az Egyesült Államok védelmi minisztériumának kutatási ügynöksége) 1988 óta közel félmillió biztonsági incidensről érkezett bejelentés.



Kormányzati Eseménykezelő Központ  
[www.cert-hungary.gov](http://www.cert-hungary.gov)

156

- ▶ **Common Vulnerabilities and Exposures (CVE)**, Minden egyes sérülékenységet egy szabványos számmal lát el pl.: CVE-2007-0038: a kernel32.dll LoadAnilcon metódusában lévő kritikus WinXP buffer overflow hiba. ([www.cve.mitre.org](http://www.cve.mitre.org))

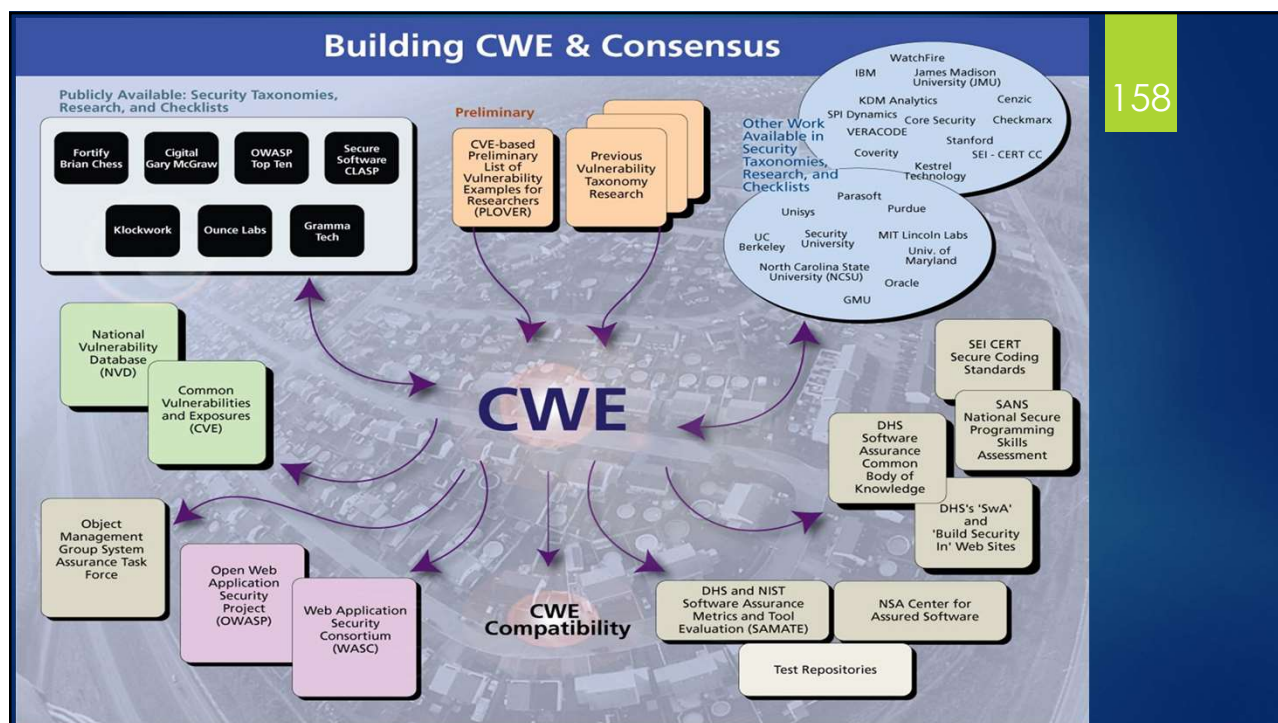
157

- ▶ **Common Weakness Enumeration (CWE)**, Nemzetközi nyílt egységesített gyűjteménye a szoftver sérülékenységeknek. Igyekszik hatékonyan támogatni a sérülékenységeket elkerülő architektúrákat és tervezést. ([www.cwe.mitre.org](http://www.cwe.mitre.org))



157





158

## 5.2. Sérülékenység fajták a CVE adatbázis alapján

- ▶ Szolgáltatás megtagadás (DoS, Slow-Dos, Distributed DoS)
- ▶ Kódvégrehajtás (saját kód futtatása, kalkulátor megnyitása)
- ▶ Túlsordulás
- ▶ Cross Site Scripting (XSS)
- ▶ Könyvtár átjárás
- ▶ Valami megkerülése
- ▶ Információ szerzés
- ▶ Jogosultság szerzés
- ▶ SQL injekció
- ▶ Fájl beszúrás
- ▶ Memória korruptió
- ▶ Cross Site Request Forgery (CSRF)
- ▶ HTTP válasz szétoztás

159

## Információ szerzés / Információ szivárgás

160

- ▶ A szoftver a lehető legkevesebb adatot szolgáltatassa! A felhasználót nem érintő (felesleges) adat ne kerüljön ki!
- ▶ Gyakori példák:
  - ❖ szolgáltatás, operációs rendszer verziószám kiszivárgás
  - ❖ belső fájlstruktúra kitakarása
  - ❖ beszédes hibaüzenet belépő felületnél
  - ❖ beszédes hibaüzenet hibás input adatra
  - ❖ belső kommentek kiszivárgása
  - ❖ stb.

160

## Példa: felhasználói névre vonatkozó információ a wordpress-ben

161

The diagram illustrates a security vulnerability in WordPress. On the left, a login form displays an error message: "ERROR: The password you entered for the username admin is incorrect. [Lost your password?](#)" The username field contains the text "admin". On the right, the same login form displays a different error message: "ERROR: Invalid username. [Lost your password?](#)". The username field is empty. A double-headed green arrow connects the two forms, indicating that the error message reveals the existence of the "admin" user.

**A hibaüzenetek csak a minimális információt tartalmazzák!**  
 (nem kell tudni, hogy van admin user)  
 Tipikus próbálkozás: admin/admin

161

## Példa: beszédes, „fecsegő” hibaüzenetek

162



Webroot  
kitakarása

Hiba: You have an error in your SQL syntax; c

Webszerver  
verzió kitakarás

|                                                |                   |      |
|------------------------------------------------|-------------------|------|
| aszkronmot meres/                              | 16-Apr-2009 14:19 | -    |
| aszkron motor példák.pdf                       | 26-May-2014 19:06 | 675K |
| motoros példák.pdf                             | 23-Jan-2015 09:33 | 897K |
| példák.DOC                                     | 30-Mar-2011 12:29 | 911K |
| Apache/2.2.22 (Ubuntu) Server at siva. Port 80 |                   |      |

162

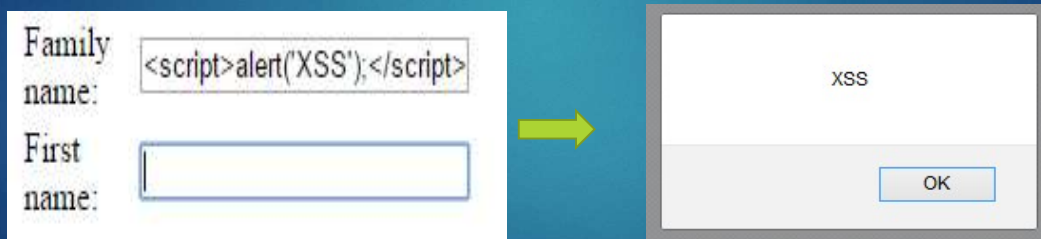
## Cross Site Scripting (XSS)

163

Miért nem jó az alábbi php kód?

```
<?php if (isset($_POST["vnev"])) { print("Üdvözlöm ".$_POST["vnev"]."!"); } ?>
```

A script bármely input adatot elfogad ellenőrzés nélkül, input validáció mindig szükséges!



A hibát kiaknázva kliens oldali kódot futtathat a böngésző !

163



## Cross Site Request Forgery

164

Minden művelet előtt jogosultság ellenőrzésnek kell lennie, amennyiben az ügyfélnek nincs session-je, az alábbi művelet nem hajtódik végre:

<http://bank.com/transfer.do?acct=BOB&amount=100>

A támadó szétküldi az alábbi linket az áldozatainak:

<http://bank.com/transfer.do?acct=ATTACKER&amount=100000>

Azok az áldozatok, akik éppen be voltak jelentkezve a banki rendszerben (van aktív session) végrehajtják az utalást!

164

## SQL Injekció

165

Miért nem jó az alábbi php kód?

```
$connect = @mysql_connect ($host, $user, $pass);
@mysql_select_db($database,$connect) or die( "Unable to select database");
if ( $connect ) { $result = mysql_query("SELECT * FROM Tabla1
Where email='".$_GET["email"]."'"); }
```

Hiányzik az input validáció, a script bármely input adatot ellenőrzés nélkül betesz az SQL lekérdezésbe!

165

## Példa SQL Injekcióra

166

bejelentkezés

Name:

Password:

Igaz/hamis lekérdezés

n of the webpage  
f the webpage

Fájl feltöltés

of the webpage  
the webpage

↑  
Php script ide

166

## Memória korrupció

167

Stack túlszordulás:

```
#include <string.h>
void func1(char* ar1)
{ char ar2[10];strcpy(ar2,ar1); }
```

```
int main(int argc, char* argv[])
{ func1(argv[1]); return 0; }
```

Amennyiben az input adat nagyobb mint 10 karakter, a func1 metódus visszatérési címe a stacken felülíródik!

167

## Példák memória korrupcióra

168

- ▶ veszélyes olyan metódusok használata, amelyek nem végeznek hossz validációt pl. strcpy, gets, puts, stb...
- ▶ tömbmásolás méretellenőrzés nélkül
- ▶ túl kevés paraméter vagy ellenőrzés nélküli input adat használat formázott stringekben, pl.: printf(„valami %u valami”); a %u miatt egy változónak kellene lennie a string után: printf(„valami %u valami”,x);
- ▶ objektumok használata a felszabadításuk után:  
free(a); a->b = 5; (Use after free)
- ▶ objektumok kétszeri felszabadítása (Double free):  
if (feltétel1) free(a); if (feltétel2) free(a);

168

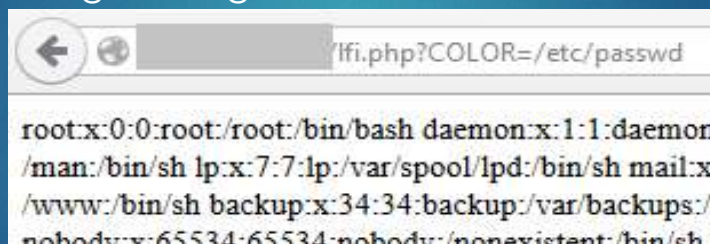
## Könyvtár átjárás

169

Miért nem jó az alábbi php kód?

```
<?php include($_GET["COLOR"]); ?>
```

A script bármely input adatot elfogad ellenőrzés nélkül, input validáció mindig szükséges!



A bemeneti adat ellenőrzésének hiánya miatt könyvtár átjárás lehetséges!

169

## Fájl beszúrás

170

Fájlfeltöltés nem megfelelő ellenőrzés miatt, pl:

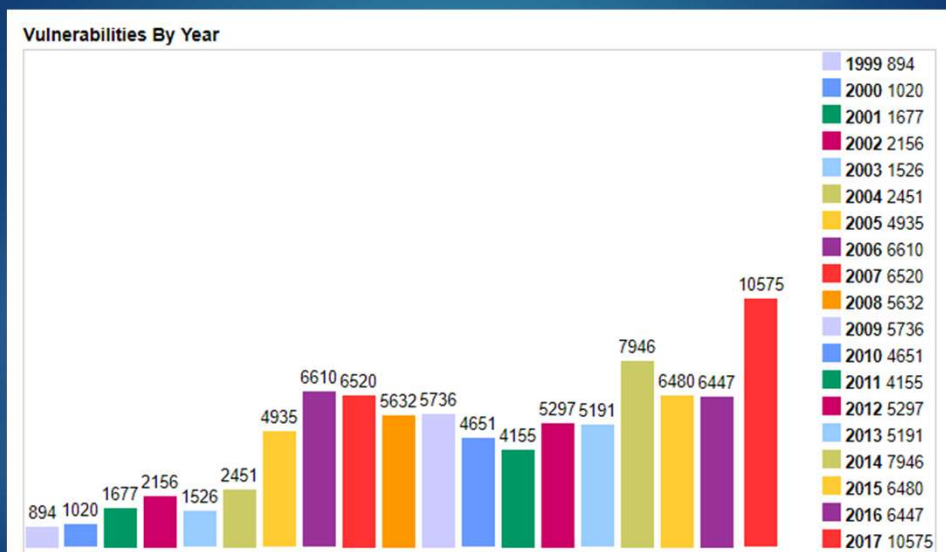
- ▶ futtatható fájlok feltöltése ellenőrzés nélkül
- ▶ futtatható fájlok feltöltése más formátummal: tamadokod.jpg
- ▶ kettős kiterjesztés: tamadokod.jpg.php
- ▶ fájlfeltöltés sql utasítással (insert into outfile)
- ▶ Stb.



170

## Sérülékenységek megoszlása évek szerint

171

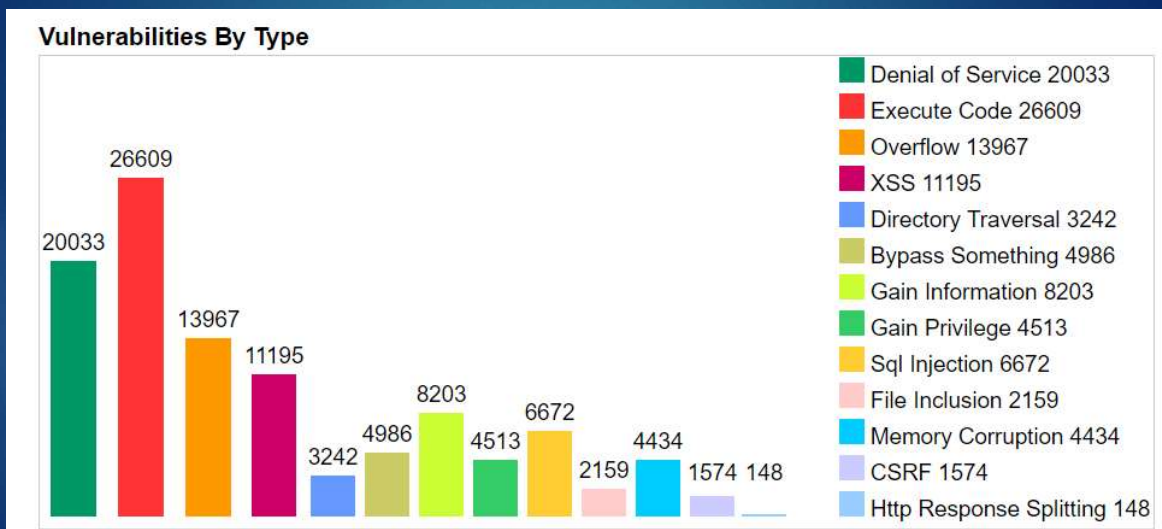


Forrás: <https://www.cvedetails.com/browse-by-date.php>

171

## Sérülékenységek megoszlása típus szerint

172



Forrás: <https://www.cvedetails.com/browse-by-types.php>

172

## 5.3. A Microsoft SDL módszertan

173

- ▶ A Microsoft Security Development Lifecycle egy speciális szoftverfejlesztési folyamat.
- ▶ Jellemzője, hogy növeli a szoftverek megbízhatóságát, különös tekintettel a biztonsági hibákra.
- ▶ 2004. januárjában készült el az első verzió, jelenleg a 2012-ben kiadott 5.2-es verzió érvényes

173



## Biztonsági koncepciók

174

- ▶ Tervezés a biztonságra
- ▶ Támadási felület csökkentése
- ▶ Mélységi védekezés
- ▶ Legkisebb privilégium elve
- ▶ Biztonságos default beállítások

Alapvető

- ▶ Biztonságos architektúrák
- ▶ Felhasználó felület tervezés
- ▶ Részletes biztonsági megfontolások
- ▶ Válaszadás biztonsági esetekre
- ▶ Speciális veszélycsökkentő eljárások implementálása

További

174

## A Microsoft SDL módszertan lépései

175

0. szakasz: Oktatás (biztonsági problémák, megoldások ismerete)

1. szakasz: Követelményspecifikáció

2. szakasz: Tervezés

3. szakasz: Implementáció

4. szakasz: Verifikáció

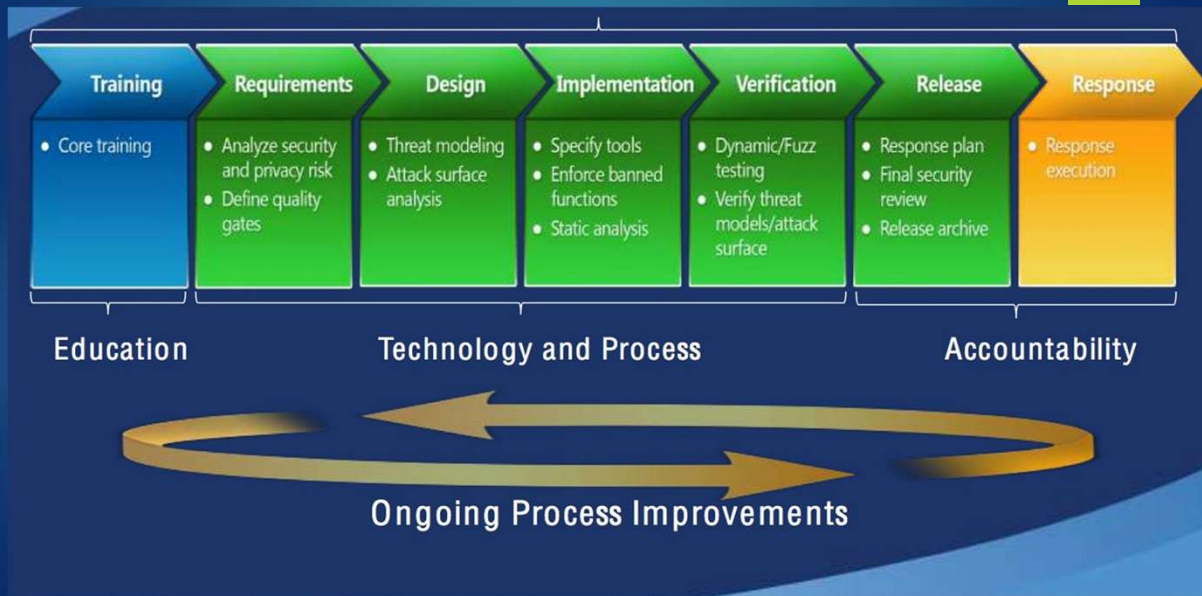
5. szakasz: Kiadás

6. szakasz: Utóélet

175

## A Microsoft SDL életrciklus modellje

176



176

## 0. szakasz: Oktatás

177

- ▶ A szoftverfejlesztésben résztvevő minden kolléga részvétele (fejlesztő, tesztelő, manager)
- ▶ Biztonsági problémák alapjai
- ▶ Legújabb biztonsági problémák
- ▶ Legújabb megoldások

177

## 1. szakasz: Követelményspecifikáció

178

- ▶ Biztonsági követelmények
- ▶ Bizalmassági követelmények
- ▶ Biztonsági ajánlások
- ▶ Erőforrások

### Biztonsági követelmények

- ▶ Szükséges-e SDL
- ▶ Biztonsági kérdések követésében felelős személy kijelölése
- ▶ Bug reporting – biztonsági problémák is kerüljenek bele
- ▶ Minimális szint kijelölése biztonság szempontjából

178

### Bizalmassági követelmények:

179

- ▶ Bizalmas adatok kezeléséért felelős kontakt személy
- ▶ Bizalmas adatokkal kapcsolatos kérdések dokumentálása (privacy bug bar)

### Biztonsági ajánlások:

- ▶ Oktatás
- ▶ Fenyegetés modellezés
- ▶ Végző biztonsági review

179

## Security bug effektus (STRIDE módszer)

180

### Security threats:

|                                 |                          |
|---------------------------------|--------------------------|
| <b>S</b> poofing :              | átverés                  |
| <b>T</b> ampering:              | babrálás                 |
| <b>R</b> epudiation:            | megtagadás               |
| <b>I</b> nformation disclosure: | információ szivárgás     |
| <b>D</b> enial of service:      | szolgáltatás megtagadása |
| <b>E</b> levation of privilege: | jogosultság emelés       |

180

## A security bug okai

181

- ▶ Buffer overflow (puffer túlcsordulás)
- ▶ Aritmetikai hiba
- ▶ Sql/script injection
- ▶ Directory traversal (könyvtár átjárás)
- ▶ Race condition
- ▶ XSS
- ▶ Kriptográfiai hiba
- ▶ Authentikációs hiba
- ▶ Authorizációs hiba
- ▶ Korlátlan erőforrás foglалás
- ▶ Hibaüzenet hibája
- ▶ Egyéb

181

## 2. szakasz: Tervezés

182

- ▶ Biztonságos architektúra
- ▶ Támadási felület minimalizálása
- ▶ Legkisebb jogosultsági szintek
- ▶ A default installáció biztonságos legyen
- ▶ Mélységi védekezés
- ▶ Új verzió esetén régi hibáinak figyelembe vétele
- ▶ Nem szükséges funkciók kivétele
- ▶ Hibaüzenetek csak a szükséges adatokat adják meg
- ▶ Ahol lehet menedzselt kódot kell használni
- ▶ Session menedzsment

182

## 3. szakasz: Implementáció

183

- ▶ Minimális kódgenerálás
- ▶ Kód review eszközök használata
- ▶ Tiltott string kezelő eljárások kerülése
- ▶ Input validáció, output encoding
- ▶ Ad-hoc sql query-k kerülése
- ▶ Web szervizek kezelése megfelelő xml parse-lal
- ▶ /GS flag használata

183



## (folytatás)

184

- ▶ ASLR használata (Address Space Layout Randomization) engedélyezni kell minden natív kód esetében.
- ▶ Biztonságos adatbáziselérés
- ▶ LINQ ExecuteQuery kerülése
- ▶ Exec kerülése a tárolt eljárásokban
- ▶ Biztonságos cookie-k használata
- ▶ Biztonságos dll betöltés
- ▶ Biztonságos redirect

184

## 4. szakasz: Verifikáció

185

- ▶ Fuzzerek használata (RPC fuzzer, ActiveX fuzzer, stb.)
- ▶ Hibák dokumentálása
- ▶ Webszervízeknél xml parser teszt
- ▶ Memória korrupciós tesztek
- ▶ Hálózati fuzzolás
- ▶ Bináris vizsgálata

185

## Tesztelés

186

- ▶ Tesztterv készítése
- ▶ Penetrációs teszt
- ▶ Regressziós sérülékenységi teszt
- ▶ Authentikációs tesztek
- ▶ Visszajátszásos tesztek
- ▶ Kód review

186

## 5. szakasz: Kiadás

187

- ▶ Bizalmasság review
- ▶ Biztonsági felelős kijelölése a kiadás utánra
- ▶ Eljárás kidolgozása biztonsági problémákra
- ▶ Trace és debug letiltása
- ▶ Végző biztonsági review

187